

IT3280 – THỰC HÀNH KIẾN TRÚC MÁY TÍNH

BÁO CÁO THỰC HÀNH

Bài 11: Lập trình xử lý ngắt – Phần 2

Họ và tên: Nguyễn Hồng Khôi

MSSV: 202416956

Assignment 4:

Chương trình:

```
.eqv IN_ADDRESS_HEXKEYBOARD 0xFFFF0012
.eqv TIMER_NOW 0xFFFF0018
.eqv TIMER_CMP 0xFFFF0020
.eqv MASK_CAUSE_TIMER 4
.eqv MASK_CAUSE_KEYPAD 8

.data
msg_keypad: .asciz "Someone has pressed a key!\n"
msg_timer: .asciz "Time interval!\n"
# -----

# MAIN Procedure
# -----

.text
main:
la t0, handler
csrrs zero, utvec, t0
li t1, 0x100
csrrs zero, uie, t1 # uie - ueie bit (bit 8) - external interrupt
csrrsi zero, uie, 0x10 # uie - utie bit (bit 4) - timer interrupt
csrrsi zero, ustatus, 1 # ustatus - enable uie - global interrupt
# -----

# Enable interrupts you expect
# -----

# Enable the interrupt of keypad of Digital Lab Sim
```

#(1)

```

li t1, IN_ADDRESS_HEX_KEYBOARD
li t2, 0x80 # bit 7 of = 1 to enable interrupt
sb t2, 0(t1)                                     #(2)
# Enable the timer interrupt
li t1, TIMER_CMP
li t2, 1000
sw t2, 0(t1)                                     #(3)
# -----
# No-end loop, main program, to demo the effective of interrupt
# -----
loop:
nop
nop
nop
j loop
end_main:
# -----
# Interrupt service routine
# -----
handler:
# Saves the context
addi sp, sp, -16
sw a0, 0(sp)
sw a1, 4(sp)
sw a2, 8(sp)

```

```

sw a7, 12(sp)
# Handles the interrupt
csrr a1, ucause
li a2, 0x7FFFFFFF
and a1, a1, a2 # Clear interrupt bit to get the value # (4)
li a2, MASK_CAUSE_TIMER
beq a1, a2, timer_isr # (5)
li a2, MASK_CAUSE_KEYPAD
beq a1, a2, keypad_isr # (6)
j end_process
timer_isr:
li a7, 4
la a0, msg_timer
ecall
# Set cmp to time + 1000
li a0, TIMER_NOW
lw a1, 0(a0)
addi a1, a1, 1000
li a0, TIMER_CMP
sw a1, 0(a0)
j end_process
keypad_isr:
li a7, 4
la a0, msg_keypad
ecall

```

```
j end_process
```

```
end_process:
```

```
# Restores the context
```

```
lw a7, 12(sp)
```

```
lw a2, 8(sp)
```

```
lw a1, 4(sp)
```

```
lw a0, 0(sp)
```

```
addi sp, sp, 16
```

```
uret
```

#(7)

Stt	Vị trí	t0	utvec	t1	uie	ustatus	t2	pc	Ghi chú
1	1	0x0040004c	0x0040004c	0x00000100	0x110	0x1	0x000	0x00400000 → 0x00400018	Thực hiện lấy địa chỉ của hàm handler và đưa vào thanh ghi utvec để thiết lập xử lý ngắt, bật ngắt thời gian bật ngắt ngoài, bật ngắt toàn cục
2	2	0x0040004c	0x0040004c	0xffff0012	0x110	0x1	0x080	0x0040001c → 0x00400028	Bật bit 7 của bàn phím để bật ngắt khi bấm phím.
3	3	0x0040004c	0x0040004c	0xffff0020	0x110	0x1	0x3e8	0x0040002c → 0x00400038	Bật ngắt cho timer, đặt thời điểm sinh ngắt đầu tiên

STT	Vị trí	a1	a2	pc	Ghi chú
1	4	0x80000004	0x7fffffff	0x0040004c đến 0x0040006c	Cất a0, a1, a2, a7 vào stack. Xác định loại ngắt thông qua thanh ghi ucause, lọc bỏ bit 31 bằng mặt nạ bit 0x7fffffff
2	5	0x00000004	0x00000004	0x00400070 0x00400074	Nếu a1=0x00000004 thì là ngắt timer
3	6	0x00000008	0x00000008	0x00400078 0x0040007c	Nếu a1=0x00000008 thì là ngắt keypad
4	7	0x00000000	0x00000000	0x004000c8 đến 0x004000dc	Khôi phục lại các thanh ghi a0, a1, a2, a7

Mô tả thuật toán:

Khởi tạo ngắt (trong main):

- Thiết lập handler làm trình phục vụ ngắt thông qua thanh ghi utvec.

Chờ ngắt (vòng loop):

- Chương trình chính chỉ chạy vòng lặp vô hạn j loop để chờ sự kiện ngắt.

Trình phục vụ ngắt (handler):

- Lưu ngữ cảnh** (các thanh ghi a0–a2, a7).
- Đọc mã nguyên nhân ngắt từ ucause, loại bỏ bit báo ngắt (bit cao nhất).
- So sánh với:
 - MASK_CAUSE_TIMER (4): nếu đúng, gọi timer_isr.
 - MASK_CAUSE_KEYPAD (8): nếu đúng, gọi keypad_isr.
- timer_isr**: in thông báo "Time interval!", cộng thêm 1000 vào thời gian hiện tại rồi cập nhật TIMER_CMP → đặt mốc ngắt tiếp theo.
- keypad_isr**: in thông báo "Someone has pressed a key!".
- Khôi phục ngữ cảnh** và trở lại chương trình chính với uret.

Assignment 5:

Chương trình:

```

.data
message: .asciz "Exception occurred.\n"

.text

main:
try:
    la t0, catch
    csrrw zero, utvec, t0 # Set utvec to the handler address
    csrrsi zero, ustatus, 1 # Set interrupt enable bit in ustatus
    lw zero, 0 # Trigger trap for Load access fault # (1)
finally:
    li a7, 10 # Exit the program
    ecall # (4)

catch:
    # Show message
    li a7, 4
    la a0, message
    ecall # (2)

    # Since uepc contains address of the error instruction
    # Need to load finally address to uepc
    la t0, finally
    csrrw zero, uepc, t0
    uret # (3)

```

Stt	Vị trí	t0	utvec	ustatus	uepc	pc	Ghi chú
1	1	0x0040001c	0x0040001c	0x00000001	0x00000000	0x00400000 đến 0x00400010	Gán t0 bằng địa chỉ của catch, nạp giá trị của t0 vào thanh ghi utvec, bật bit uie trong thanh ghi
2	2	0x0040001c	0x0040001c	0x00000010	0x00400010	0x0040001c đến 0x00400028	In ra thông báo trong message, thanh ghi uepc chứa địa chỉ của lệnh gây lỗi
3	3	0x00400014	0x0040001c	0x00000010	0x00400014	0x0040002c đến 0x00400038	Gán t0 bằng địa chỉ của finally, uret quay lại user mode và tiếp tục từ địa chỉ finally
4	4	0x00400014	0x0040001c	0x00000001	0x00400014	0x00400014 đến 0x00400018	Nhảy đến finally, kết thúc chương trình

Mô tả thuật toán:

- Chương trình minh họa mô hình try-catch-finally, trong đó:
 - **Try** là phần chứa các lệnh có nguy cơ phát sinh lỗi.
 - **Catch** là nơi chứa mã xử lý khi lỗi xảy ra.
 - **Finally** là khối lệnh luôn được chạy, bất kể có lỗi hay không.
 - Trong khối **try**, ta thiết lập địa chỉ của nhãn **catch** (hàm xử lý lỗi) vào CSR utvec và bật ngắt toàn cục. Tiếp đó thực thi câu lệnh có thể gây lỗi: ở đây lw zero, 0 sẽ phát sinh lỗi truy cập bộ nhớ.
 - Khối **finally** chứa các lệnh dọn dẹp hoặc kết thúc chương trình, luôn được thực hiện sau khi try/catch.
 - Trong **catch**, chương trình in thông báo lỗi rồi chuyển tiếp điều khiển về khối finally.

a. Lưu trạng thái trước khi xử lý: Lưu các thanh ghi a0 và a7 vào stack để không mất dữ liệu khi thực hiện xử lý ngắt.

Assignment 6:

.data

message: .asciz "OVERFLOW OCCURRED\n"

.text

main:

la t0, handler # (1) Thiết lập handler (utvec) và bật UIE

csrrs zero, utvec, t0

csrrsi zero, ustatus, 1

check:

li s1, 0x7fffffff # (2) Nạp toán hạng

li s2, 0x1fffffff

add s3, s1, s2 # (3) Tính tổng $s3 = s1 + s2$ và kiểm tra tràn

xor t1, s1, s2

blt t1, zero, EXIT

slt t2, s3, s1

blt s1, zero, NEGATIVE

beq t2, zero, EXIT

j OVERFLOW

NEGATIVE:

bne t2, zero, EXIT # s1 âm, không tràn nếu $s3 \geq s1$

j OVERFLOW

OVERFLOW:

```
li    t1, 1          # (4) Bật ngắt mềm (uip bit 0) và in thông báo
csrrs zero, uip, t1
ecall
```

EXIT:

```
li    a7, 10         # (5) Thoát chương trình
ecall
```

handler:

```
addi  sp, sp, -20     # Lưu ngữ cảnh
sw    a0, 0(sp)
sw    a7, 4(sp)
sw    s1, 8(sp)
sw    s2, 12(sp)
sw    s3, 16(sp)
```

```
li    a7, 4           # In thông báo lỗi
la    a0, message
ecall
```

```
lw    s3, 16(sp)      # Khôi phục ngữ cảnh
lw    s2, 12(sp)
lw    s1, 8(sp)
```

lw a7, 4(sp)

lw a0, 0(sp)

addi sp, sp, 20

j EXIT # Về EXIT để gọi ecall exit

STT	Bước	utvec	ustatus	uip	t1	PC trước→sau	Ghi chú
1	(1)	0x00400058	0x1	0x0	0x00000000	0x00400000→0x004000c	Thiết lập utvec & UIE
2	(2)	0x00400058	0x1	0x0	0x00000000	0x004000c→0x00400018	Nạp toán hạng
3	(3)	0x00400058	0x1	0x0	0x60000000	0x00400018→0x00400040	Tính tổng & phát hiện tràn
4	(4)	0x00400058	0x1	0x1	0x60000000	0x00400040→0x0040004c	Bật uip & in thông báo
5	(5)	0x00400058	0x1	0x1	0x60000000	0x0040004c→0x00400054	Thoát chương trình

Kết quả:

D:\[Tài liệu học tập]\Thực hành KTM\Lab11\Assignment6.asm - RARS 1.6

File Edit Run Settings Tools Help

Run speed at max (no interaction)

Execute

Text Segment

Addr	Code	Basic	Source
0x00400000	0x00000297	auipc x5,0	6: la t0, handler # (1) T...
0x00400004	0x05828293	addi x5,x5,0x00000058	
0x00400008	0x0052a073	crrrs x0,5,x5	7: crrrs zero, utvec, t0
0x0040000c	0x0000e073	crrrsi x0,0,1	8: crrrsi zero, ustatus, 1
0x00400010	0x800004b7	lui x9,0x00080000	11: li s1, 0x7fffffff # (2) N...
0x00400014	0xffff4849	addi x5,x5,0xffffffff	
0x00400018	0x20000937	lui x18,0x00020000	12: li s2, 0xffffffff
0x0040001c	0xff909513	addi x18,x18,0xffffffff	
0x00400020	0x012498b3	add x19,x9,x18	14: add s3,s1,s2 # (3) T...
0x00400024	0x0124c337	xor x6,x9,x18	15: xor t1,s1,s2
0x00400028	0x02034463	blt x6,x0,0x00000028	16: blt t1,zero,EXIT
0x00400030	0x0086a3b3	slr x6,x3,x18,x18	17: slr s3,s3,s1

Data Segment

Address	Value (+0)	Value (+4)	Value (+8)	Value (+c)	Value (+10)	Value (+14)	Value (+18)	Value (+1c)
0x10010000	0x5245564e	0x574f4e46	0x43484e20	0x55252555	0x00000044	0x00000000	0x00000000	0x00000000
0x10010020	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010040	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010060	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010080	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100a0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100c0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x100100e0	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000
0x10010100	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000	0x00000000

Labels

Label	Address
main	0x00400000
check	0x00400010
NEGATIVE	0x0040003c
OVERFLOW	0x00400044
EXIT	0x00400050
handler	0x00400058
message	0x10010000

Registers

Name	Number	Value
zero	0	0x00000000
ra	1	0x00000000
sp	2	0x7fffffff
fp	3	0x10008000
tp	4	0x00000000
t0	5	0x00400058
t1	6	0x00000001
t2	7	0x00000001
s0	8	0x00000000
s1	9	0x7fffffff
a0	10	0x00000000
a1	11	0x00000000
a2	12	0x00000000
a3	13	0x00000000
a4	14	0x00000000
a5	15	0x00000000
a6	16	0x00000000
a7	17	0x0000000a
s2	18	0xffffffff
s3	19	0x9fffffff
s4	20	0x00000000
s5	21	0x00000000
s6	22	0x00000000
s7	23	0x00000000
s8	24	0x00000000
s9	25	0x00000000
s10	26	0x00000000
s11	27	0x00000000
t3	28	0x00000000
t4	29	0x00000000
t5	30	0x00000000
t6	31	0x00000000
pc		0x00400058

Messages

Run I/O

OVERFLOW OCCURRED

-- program is finished running (0) --

Clear

Activate Windows
Go to Settings to activate Windows.

Mô tả:

Khởi ngắt:

Thiết lập handler làm vector ngắt và bật UIE để cho phép ngắt phần mềm.

Nạp toán hạng:

Đưa hai giá trị cực đại vào thanh ghi s1 và s2.

Tính tổng & kiểm tra tràn:

- Tính $s3 = s1 + s2$.
- Dùng xor kiểm tra nếu hai toán hạng khác dấu $\rightarrow$ không tràn.
- Dùng slt kiểm tra nếu tổng nhỏ hơn một trong hai toán hạng cùng dấu $\rightarrow$ tràn.

Gây ngắt & in thông báo:

Nếu phát hiện tràn, bật bit ngắt mềm (uip) và gọi ecall để in “OVERFLOW OCCURRED”.

Kết thúc:

Trong ISR, in lỗi, khôi phục ngữ cảnh rồi thoát chương trình (ecall exit).

Assignment bonus:

- Ghép nối đồng thời Digital Lab Simulator và Timer Tool.
- Bắt sự kiện nhấn nút bàn phím Keypad 4x4 và sự kiện timer compare bằng ngắt.
- Khi chương trình bắt đầu chạy, hiển thị giá trị tăng dần từ 00 đến 99 trên 2 đèn LED 7 thanh. Chu kỳ được đặt mặc định bằng 1000.
- Xử lý khi bấm nút trên Keypad:
 - Nút 0 được bấm: chế độ đếm tăng dần.
 - Nút 1 được bấm: chế độ đếm giảm dần.
 - Nút 4 được bấm: giảm chu kỳ đếm (tăng tốc độ đếm).
 - Nút 5 được bấm: tăng chu kỳ đếm (giảm tốc độ đếm).

Chương trình:

```
.eqv IN_ADDRESS_HEX_A_KEYBOARD 0xFFFF0012 # địa chỉ thanh ghi ngắt bàn phím
```

```
.eqv OUT_ADDRESS_HEX_A_KEYBOARD 0xFFFF0014
```

```
.eqv TIMER_NOW 0xFFFF0018           # địa chỉ thanh ghi thời gian hiện tại
```

```
.eqv TIMER_CMP 0xFFFF0020          # địa chỉ thanh ghi so sánh thời gian
```

```
.eqv MASK_CAUSE_TIMER 4             # mã nguyên nhân ngắt timer
```

```
.eqv MASK_CAUSE_KEYPAD 8            # mã nguyên nhân ngắt bàn phím
```

```
.eqv SEVENSEG_LEFT 0xFFFF0011
```

```
.eqv SEVENSEG_RIGHT 0xFFFF0010      # địa chỉ hai đèn LED
```

```
.data
```

```
msg_tang:      .asciz "Dem tang dan.\n"
```

```
msg_giam:      .asciz "Dem giam dan.\n"
```

```
msg_giamtoc:   .asciz "Giam toc. "
```

```
msg_tangtoc:   .asciz "Tang toc. "
```

```
thong_bao_thoi_gian:.asciz "Thoi gian 1 chu ky hien tai la: "
```

```
msg_ms:        .asciz " ms.\n"
```

```
ma_led:        .word 0x3F 0x06 0x5B 0x4F 0x66 0x6D 0x7D 0x07 0x7F 0x6F
```

```
bien_dem:      .word 0
```

```
chu_ky:        .word 1000
```

```
huong_dem:     .word 0           # 0 = tang, 1 = giam
```

```
.text
```

main:

```
la    t0, handler
```

```
csrrs zero, utvec, t0
```

```
li    t1, 0x100
```

```
csrrs zero, uie, t1
```

```
csrrsi zero, uie, 0x10
```

```
csrrsi zero, ustatus, 1
```

```
li    t1, IN_ADDRESS_HEX_A_KEYBOARD
```

```
li    t2, 0x80
```

```
sb    t2, 0(t1)
```

```
li    t1, TIMER_CMP
```

```
li    t2, 1000
```

```
sw    t2, 0(t1)
```

loop:

```
nop
```

```
nop
```

```
nop
```

```
j     loop
```

handler:

```
csrr  a1, ucause
```

```
li    a2, 0x7FFFFFFF
```

```
and   a1, a1, a2
```

```
li    a2, MASK_CAUSE_TIMER
```

```
beq   a1, a2, xlngat_timer
```

```
li    a2, MASK_CAUSE_KEYPAD
```

```
beq   a1, a2, xlngat_banphim
```

```
j     end_handler
```

Xử lý ngắt timer

xlngat_timer:

```
la    t1, bien_dem
```

```
lw    t2, 0(t1)
```

```
li    t1, 10
```

```
div   t3, t2, t1
```

```
rem   t4, t2, t1
```

```
la    t1, ma_led
```

```
slli  t3, t3, 2
```

```
add   t3, t3, t1
```

```
lw    t5, 0(t3)
```

```
li    t6, SEVENSEG_LEFT
```

```
sb    t5, 0(t6)
```

```
slli    t4, t4, 2
add     t4, t4, t1
lw      t5, 0(t4)
li      t6, SEVENSEG_RIGHT
sb      t5, 0(t6)
```

```
la      t1, huong_dem
lw      t3, 0(t1)
beq     t3, zero, tang_dem
```

giam_dem:

```
addi    t2, t2, -1
li      t4, 0
bge     t2, t4, cap_nhat_dem
li      t2, 99
j       cap_nhat_dem
```

tang_dem:

```
addi    t2, t2, 1
li      t4, 99
ble     t2, t4, cap_nhat_dem
li      t2, 0
j       cap_nhat_dem
```

cap_nhat_dem:


```
la    t1, bien_dem
```

```
sw    t2, 0(t1)
```

```
# cập nhật chu kỳ
```

```
li    t1, TIMER_NOW
```

```
lw    t2, 0(t1)
```

```
la    t3, chu_ky
```

```
lw    t4, 0(t3)
```

```
add   t2, t2, t4
```

```
li    t1, TIMER_CMP
```

```
sw    t2, 0(t1)
```

```
j     end_handler
```

```
# Xử lý ngắt bàn phím
```

```
xlnгат_banphim:
```

```
li    t1, IN_ADDRESS_HEXА_KEYBOARD
```

```
li    t2, 0x81      # Bật ngắt của bàn phím và quét hàng 1
```

```
sb    t2, 0(t1)
```

```
li    t1, OUT_ADDRESS_HEXА_KEYBOARD
```

```
lb    t2, 0(t1)
```

```
li    t3, 0x11
```

```
beq   t2, t3, chon_tang
```

```
li    t3, 0x21
beq    t2, t3, chon_giam
```

```
li    t1, IN_ADDRESS_HEX_KEYBOARD
li    t2, 0x82    # Bật ngắt của bàn phím và quét hàng 2
sb     t2, 0(t1)
li    t1, OUT_ADDRESS_HEX_KEYBOARD
lb     t2, 0(t1)
```

```
li    t3, 0x12
beq    t2, t3, chon_tangtoc
```

```
li    t3, 0x22
beq    t2, t3, chon_giamtoc
```

```
j      end_handler
```

chon_tang:

```
la     t1, huong_dem
li     t2, 0
sw     t2, 0(t1)
li     a7, 4
la     a0, msg_tang
ecall
j      end_handler
```

chon_giam:

la t1, huong_dem

li t2, 1

sw t2, 0(t1)

li a7, 4

la a0, msg_giam

ecall

j end_handler

chon_giamtoc:

la t1, chu_ky

lw t2, 0(t1)

li t3, 2

mul t2, t2, t3

sw t2, 0(t1)

li a7, 4

la a0, msg_giamtoc

ecall

li a7, 4

la a0, thong_bao_thoi_gian

ecall

li a7, 1

mv a0, t2

ecall

```
li    a7, 4
la    a0, msg_ms
ecall
j     end_handler
```

chon_tangtoc:

```
la    t1, chu_ky
lw    t2, 0(t1)
li    t3, 2
div   t2, t2, t3
sw    t2, 0(t1)
li    a7, 4
la    a0, msg_tangtoc
ecall
li    a7, 4
la    a0, thong_bao_thoi_gian
ecall
li    a7, 1
mv    a0, t2
ecall
li    a7, 4
la    a0, msg_ms
ecall
j     end_handler
```

end_handler:

uret

Kết quả:

(Đã đính kèm file video vào assignment)

Mô tả:

Chương trình sử dụng **ngắt timer** để điều khiển hiển thị số từ **00 đến 99** trên **hai LED 7 đoạn** và cho phép người dùng điều khiển hướng đếm (tăng/giảm) và tốc độ đếm thông qua **bàn phím HEXA**.

1. Cơ chế hoạt động chính

- **Timer interrupt** xảy ra định kỳ (dựa trên giá trị trong thanh ghi chu_ky).
- Khi xảy ra ngắt:
 - Đọc và cập nhật giá trị biến đếm (bien_dem) theo hướng đếm (tăng hoặc giảm).
 - Hiển thị số mới lên 2 LED 7 đoạn bằng cách tách hàng chục và hàng đơn vị và tra bảng mã ma_led.
 - Cập nhật thời điểm so sánh mới (TIMER_CMP) để lập lại chu kỳ tiếp theo.

2. Xử lý bàn phím (keypad interrupt)

- Quét 2 hàng phím để kiểm tra mã phím người dùng nhấn.
- Các mã phím tương ứng các chức năng:
 - 0x11: Chuyển sang **đếm tăng**, in dòng "Dem tang dan."
 - 0x21: Chuyển sang **đếm giảm**, in dòng "Dem giam dan."
 - 0x12: **Tăng tốc đếm** (giảm chu_ky một nửa), in thời gian chu kỳ mới.
 - 0x22: **Giảm tốc đếm** (gấp đôi chu_ky), in thời gian chu kỳ mới.